

MoE Scaling on TPU with JAX Parallelism

A systems-level implementation study of Mixture-of-Experts routing, collective matmuls, and explicit communication control on TPU v5e-8 hardware.

HARDWARE

Kaggle TPU v5e-8 · 8 cores

EXECUTED

2026-03-29 · 72 s runtime

KAGGLE NOTEBOOK

reptor420/moe-scaling-on-tpu

KEY RESULT

43× MoE speedup (73 ms → 1.7 ms)

Based on:

Part 10 — All About JAX

How To Scale Your Model · Austin et al., Google DeepMind, 2025

<https://jax-ml.github.io/scaling-book/jax-stuff/>

GitHub: https://github.com/YashJayswal24/Model_scaling_jax

Contents

1	Hardware & Environment	2
2	Three JAX Parallelism Modes	2
3	Problem 1 — Sharded Average & Roll-Diff	2
3.1	Per-Shard Average	2
3.1.1	With jax.jit (reshape trick)	2
3.1.2	With shard_map (device-local view)	3
3.2	Roll-Difference via shard_map	3
4	Problem 2 — Mixture of Experts Routing	3
4.1	Dimensions	3
4.2	Naive jit MoE — 73 ms	3
4.3	Pipelined All-to-All — 1.7 ms (43× faster)	4
4.3.1	Three-Phase Pipeline	4
4.3.2	Key Communication Code	4
4.4	Top- k MoE ($k = 2$) — Zero Overhead	5
5	Problem 3 — Collective Matmuls & Overlapped MLP	5
5.1	Baseline — jax.jit Auto-Sharded MLP (1.2 ms)	5
5.2	AllReduce Matmul via shard_map	5
5.3	ReduceScatter — Three Variants	5
5.4	End-to-End Overlapped Transformer MLP (1.25 ms)	6
6	Full Performance Summary	7
7	CV-Style Achievements & Contributions	7
7.1	🏆 Performance Wins	7
7.2	📅 Work Completed (10 Tasks)	8
7.3	✂️ Code Quality Improvements	8
7.4	📐 Technical Concepts Demonstrated	9
8	Bug Fixes & API Updates	9
9	Resources & Citation	10

1 Hardware & Environment

Property	Value
Accelerator	TPU v5e-8
Devices	8 × TpuDevice, process_index=0, coords (0,0,0)→(1,3,0)
PJRT API version	0.69 (framework 0.90)
Python	3.12 (papermill execution)
Key env flag	<code>LIBTPU_INIT_ARGS=-xla_tpu_enable_async_collective_fusion=true</code>
Profiler traces	9 separate Perfetto traces (AOT-compiled)
Total runtime	72 seconds (25 cells)

Transparent hugepages not enabled. TPU v5e runtime startup/shutdown latency was degraded. Fix: `sudo sh -c "echo always > /sys/kernel/mm/transparent_hugepage/enabled"`

2 Three JAX Parallelism Modes

The book identifies three contracts between programmer and JAX runtime for multi-device execution:

Mode	API	Who decides communication?	Errors on ambiguity?
Auto	<code>jax.jit</code> (Auto axes)	XLA / Shardy compiler	No — may silently do wrong thing
Explicit	<code>jax.jit</code> (Explicit axes)	JAX type system	Yes — errors on ambiguous sharding
Manual	<code>jax.shard_map</code>	You — fully explicit	N/A — you write every byte

This notebook exercises all three modes, matching the book’s pedagogical progression. Each mode’s limitation motivates the next.

3 Problem 1 — Sharded Average & Roll-Diff

Reference: §Worked Problems, Problem 1 **Mesh:** $(x = 2, y = 4)$, `AxisType.Explicit`

3.1 Per-Shard Average

Array $A : \text{float32}[S_X, D_Y]$ sharded across the mesh. Goal: return $[X, Y]$ where each entry is the mean of shard (i, j) .

With `jax.jit` (reshape trick)

```
@jax.jit
def average(arr):
```

```
# Reshape [x, p, y, q] -> mean over local (non-sharded) dims (1, 3)
arr = jnp.mean(arr, axis=(1, 3))
return jax.lax.with_sharding_constraint(arr, P('x', 'y'))
```

With shard_map (device-local view)

```
@jax.shard_map(in_specs=jax.P('x', 'y'), out_specs=jax.P('x', 'y'))
def avg_shard_map(arr):
    assert arr.shape == (p, q)          # each device sees only its
    shard
    return jnp.mean(arr).reshape((1, 1))
```

Result: `[[4.5, 6.5, 8.5, 10.5], [20.5, 22.5, 24.5, 26.5]]` ✓ — zero communication on both.

Key insight: With `shard_map`, the absence of any `psum` / `all_gather` / `ppermute` in the function body *proves* zero communication. This is impossible to verify with `jax.jit` without reading the compiled HLO.

3.2 Roll-Difference via shard_map

```
@jax.shard_map(
    in_specs=(jax.P('x', 'y'), jax.P('x', 'y'), jax.P()),
    out_specs=jax.P('x', 'y')
)
def roll_diff(base, roll, shift):
    return base - jnp.roll(roll, shift=shift, axis=0)
# Result (shift=1): rows alternate between -8 and +8 checkmark
```

Profiler traces saved: /kaggle/working/shardmap_avg/ and /kaggle/working/roll_diff/.

4 Problem 2 — Mixture of Experts Routing

Reference: §Worked Problems, Problem 2 **Mesh:** $(x = 8, y = 1)$

4.1 Dimensions

Symbol	Value	Description
E	8	Total experts (one per device)
S	2048	Total tokens ($S_x = 256$ per device)
D	4096	Hidden dimension
F	14336	Expert output dimension (Llama-8B FFN proxy)
N	8	Number of devices
C	32	Expert capacity — tokens per communication round

4.2 Naive jit MoE — 73 ms

The natural scan-based approach iterates over all E experts with a binary mask:

```
def process_exp(carry, e):
    mask = (B == e)[: , None]    # [S, 1] -- which tokens belong to
    to_add = A @ W[e]             # [S, F] -- ALL tokens x expert e's
    return carry + to_add * mask, None # E-1 results are thrown
    away

out, _ = jax.lax.scan(process_exp, output, jnp.arange(E))
```

What XLA did: auto-sharding assigned *both* A and W along axis x (data-parallel). This forced an AllGather of all activations before the scan — $O(S \times D)$ memory replicated per device.

Observed latency: 73 ms. The AllGather alone dominates. At production scale ($S = 10^6$, $D = 8192$), this would replicate gigabytes of activations across every device on every forward pass.

4.3 Pipelined All-to-All — 1.7 ms (43× faster)

Instead of gathering all activations, use explicit **All-to-All** to route only the tokens belonging to each expert.

Three-Phase Pipeline

Phase	Name	Description
Phase 1	PreFetch	Sort tokens by expert, bin into $[E, C, D]$ chunk, fire first All-to-All $\rightarrow [E_x, N \cdot C, D]$
Phase 2	Unrolled Pipeline	Python for -loop: compute current chunk while fetching next. XLA overlaps comm + compute in the flattened graph.
Phase 3	Drain	Process last pre-fetched chunk, reverse All-to-All $\rightarrow [E, C, F]$, scatter back to $[S_x, F]$

Key Communication Code

```
# Forward dispatch: send tokens to expert-owning devices
recv_A = jax.lax.all_to_all(
    send_A, axis_name='x', split_axis=0, concat_axis=1
) # [E, C, D] -> [E_x, N*C, D]

# Dense local matmul -- zero wasted compute
recv_Out = jnp.einsum('edf,ecd->ecf', W_local, recv_A)

# Backward return: results back to origin devices
recv_Back = jax.lax.all_to_all(
    recv_Out, axis_name='x', split_axis=1, concat_axis=0
) # [E_x, N*C, F] -> [E, C, F]

# Pipelined overlap (Python loop = unrolled in XLA graph)
for i in range(num_scan_steps):
    carry_out = compute_and_scatter(recv_A_curr, valid_curr,
                                     safe_idx_curr, carry_out) #
    math first
    recv_A_curr, valid_curr, safe_idx_curr = fetch_chunk(i+1) # comm
    second
```

Observed latency: 1.7 ms. A **43×** speedup over the naive AllGather approach. AOT-compiled via `.lower(W, A, B).compile()` for clean, JIT-warmup-free profiling. Trace: `MoE_chunk_all_to_all/`.

4.4 Top- k MoE ($k = 2$) — Zero Overhead

Extension to Top-2 routing (each token processed by 2 experts, averaged):

```
flat_B = B_local.reshape(-1)           # [S_x, 2] -> [S_x * 2]
flat_A = jnp.repeat(A_local, k, axis=0) # duplicate each token k
                                         times
# ... same All-to-All pipeline ...
final_out = jnp.mean(final_out_k, axis=1) # average k expert
                                         outputs
```

Observed latency: ≈ 1.7 ms — identical to Top-1. The pipeline processes $2\times$ the tokens at the same capacity C with the same number of communication rounds.

5 Problem 3 — Collective Matmuls & Overlapped MLP

Reference: §Worked Problems, Problem 3 **Dims:** $BX = D = 4096$, $F = 14336$ (Llama-8B proxy)

5.1 Baseline — `jax.jit` Auto-Sharded MLP (1.2 ms)

```
# Mesh (x=4, y=2), AxisType.Auto
@jax.jit
def compute_mlp(In, Win, Wout):
    return jax.lax.with_sharding_constraint(In @ Win @ Wout,
                                             sharding)
# In -> P('x', 'y')   Win -> P(None, 'y')   Wout -> P('y', None)
# Profiler: jit_MLP_Auto_Shard/ -- 1.2 ms total
# AllReduce: ~0.2 ms | AllGather: ~0.1 ms
```

5.2 AllReduce Matmul via `shard_map`

Implements $A[BX, DY] \cdot W[DY, F] \rightarrow \text{Out}[BX, F]$:

```
@jax.shard_map(
    in_specs=(P('x', 'y'), P('y', None)), out_specs=P('x', None)
)
def compute_matmul_allreduce(A, W):
    return jax.lax.psum(A @ W, axis_name='y') # local matmul +
AllReduce
```

5.3 ReduceScatter — Three Variants

For the down-projection $\text{Tmp}[BX, FY] \cdot W_2[FY, D] \rightarrow \text{Out}[BX, DY]$:

Variant	Mechanism	Latency
psum_scatter	JAX built-in hardware-optimized primitive	1.01 ms ✓
Manual ring (unidirectional)	$y - 1$ ppermute steps rightward; Python loop	1.2 ms
Recursive halving (bidirectional)	$\log_2(y)$ XOR-partner exchanges; array halves each step	$\approx 1.1 \mu\text{s}$ ✓

The recursive halving pattern uses XOR to select partners — a classic butterfly all-reduce:

```
for step in reversed(range(int(math.log2(y)))):
    # XOR partner: device i exchanges with i XOR 2^step
    perm = tuple((j, j ^ (1 << step)) for j in range(y))
    recv_data = jax.lax.ppermute(send_data, axis_name='y', perm=perm)
    state = keep_data + recv_data # array shrinks by 2x each step
# After log2(y) steps: [BX, D] -> [BX, DY], fully summed and placed
```

5.4 End-to-End Overlapped Transformer MLP (1.25 ms)

Composing AllGather up-projection + recursive halving ReduceScatter down-projection with GeLU:

```
# Up-projection: AllGather along y to reconstruct full D dimension
In_gathered = jax.lax.all_gather(In_local, axis_name='y', tiled=True,
    , axis=1)
act_local = jax.nn.gelu(In_gathered @ Win_local) # [BX, FY_local]

# Down-projection: recursive halving ReduceScatter
final_out = compute_matmul_recursive_halving_rs(act_local,
    Wout_local)
# Out: [BX, DY] -- correct sharding at JIT-speed parity
```

Why no speedup? XLA already performs compute-communication overlap on TPU internally. Our manual overlap adds *transparency* — every byte is visible in Perfetto — but not raw speed. The value is correctness guarantees and communication auditability.

6 Full Performance Summary

#	Experiment	Latency	vs Baseline	Profiler Folder
1	Naive jit MoE (AllGather)	73 ms	Baseline	MoE_auto/
2	Pipelined All-to-All MoE ($k = 1$)	1.7 ms	43× faster	MoE_chunk_all_to_all/
3	Top- k MoE ($k = 2$)	≈ 1.7 ms	0% overhead	MoE_chunk_all_to_all_topk/
4	jit Auto-Sharded MLP	1.2 ms	Baseline	jit_MLP_Auto_Shard/
5	AllReduce Matmul (psum)	—	—	experimental_al_reduce/
6	ReduceScatter (psum_scatter)	1.01 ms	16% vs ring	.._psum/
7	ReduceScatter (manual ring)	1.2 ms	—	.._matmul/
8	ReduceScatter (recursive halving)	≈ 1.1 μ s	Near optimum	.._halving_rs/
9	Overlapped Transformer MLP	1.25 ms	JIT parity	transformer_mlp/

7 CV-Style Achievements & Contributions

7.1 🏆 Performance Wins

Achievement	Before	After	Result
MoE routing: AllGather $\rightarrow$ All-to-All pipeline	73 ms	1.7 ms	🏆 43× faster
ReduceScatter: manual ring vs psum_scatter	1.2 ms	1.01 ms	16% reduction
Bidirectional halving vs unidirectional ring	1.2 ms	≈ 1.1 μ s	Near optimum
Top- k MoE ($k = 2$) vs Top-1 overhead	1.7 ms	1.7 ms	0% — free scaling
Total notebook runtime on TPU v5e-8	—	72 s	25 cells, 9 Perfetto traces

7.2 📋 Work Completed (10 Tasks)

#	Task	APIs Used	Scale
1	Sharded average via <code>jax.jit</code> (reshape trick)	<code>jit</code> , <code>with_sharding_constraints</code>	8 TPUs, (2×4) mesh
2	Same average via <code>shard_map</code> — 0 comms verified	<code>shard_map</code> , <code>jnp.mean</code>	8 shards, 0 AllGather
3	Intra-shard roll-difference	<code>shard_map</code> , <code>jnp.roll</code>	shift=1, 8 shards
4	Naive MoE profiled — AllGather bottleneck found	<code>lax.scan</code> , <code>jax.profiler</code>	$E=8, S=2048, D=4096$
5	3-phase pipelined All-to-All MoE	<code>shard_map</code> , <code>lax.all_to_all</code> , AOT	$C=32$, 43× speedup
6	Top- k routing ($k=2$) with token duplication	<code>jnp.repeat</code> , reshape + mean	2× token expansion
7	AllReduce matmul via <code>psum</code>	<code>shard_map</code> , <code>lax.psum</code>	$BX=D=4096, F=14336$
8	ReduceScatter in 3 variants	<code>psum_scatter</code> , <code>ppermute</code> , <code>dynamic_slice</code>	4×2 mesh, 4-way ring
9	Overlapped Transformer MLP with GeLU	<code>all_gather</code> , recursive RS, <code>gelu</code>	Full Llama-8B proxy dims
10	9 AOT-compiled Perfetto traces	<code>.lower().compile()</code> , <code>profiler.trace</code>	All runs JIT-warmup free

7.3 ✂ Code Quality Improvements

Severity	Issue	Fix	Risk If Ignored
Critical	<code>LIBTPU_INIT_ARGS</code> set twice — 2nd silently kills 1st	Combine into single space-separated string	<code>async_all_to_all</code> never active during entire run
API	<code>jax.lax.pvary</code> deprecated (2 cells)	<code>jax.lax.pcast(..., to='varying')</code>	Breaks on next JAX upgrade
UX	8 terse markdown headers (e.g. "part 1")	Replace with ## headings + results tables	Notebook unreadable when shared
Missing	No notebook title or citation cell	Add header cell with book link	No attribution or reproducibility

7.4 📖 Technical Concepts Demonstrated

Concept	Where Used	Book Section
All 3 JAX parallelism modes	Problems 1, 2, 3	§How Does Parallelism Work
<code>shard_map</code> explicit communication	6 of 10 experiments	Manual sharding mode
All 4 collective types (AR, AG, RS, A2A)	Throughout	Part 10 Collective examples
Expert Parallelism with capacity buckets	MoE All-to-All pipeline	Problem 2 solution
Compute-comm overlap (unrolled Python loops)	MoE pipeline + ring RS	Collective Matmul example
Bidirectional butterfly ($\log_2 N$ steps)	Recursive halving RS	Problem 4
AOT compilation + Perfetto tracing	All 9 profiler runs	Part 9 — Profiling
Llama-8B proxy dims ($D=4096, F=14336$)	Problem 3	Applied Training

8 Bug Fixes & API Updates

```
# === BUG: Second assignment silently kills first ===
os.environ["LIBTPU_INIT_ARGS"] = "--xla_tpu_enable_async_all_to_all=true"
os.environ["LIBTPU_INIT_ARGS"] = "--
    xla_tpu_enable_async_collective_fusion=true"
# async_all_to_all was NEVER enabled during the entire run!

# === FIX: Combine into one string ===
os.environ["LIBTPU_INIT_ARGS"] = (
    "--xla_tpu_enable_async_all_to_all=true "
    "--xla_tpu_enable_async_collective_fusion=true"
)

# === DEPRECATED API ===
initial_out = jax.lax.pvary(initial_out, 'x') # DEPRECATED

# === CURRENT API ===
initial_out = jax.lax.pcast(initial_out, to='varying') # CORRECT
```

9 Resources & Citation

Resource	Link
Kaggle Notebook	https://www.kaggle.com/code/reptor420/moe-scaling-on-tpu
Implementation Folder	https://github.com/YashJayswal24/Model_scaling_jax/tree/main/10_jax_parallelism_moe
JAX Scaling Book (Part 10)	https://jax-ml.github.io/scaling-book/jax-stuff/
Full Repo	https://github.com/YashJayswal24/Model_scaling_jax
Portfolio Blog Post	https://yashjayswal24.github.io/portfolio/blog/2026/jax-parallelism-moe

```
@article{scaling-book,
  title      = {How to Scale Your Model},
  author     = {Austin, Jacob and Douglas, Sholto and Frostig, Roy and
               Levskaya, Anselm and Chen, Charlie and Vikram, Sharad and
               Lebron, Federico and Choy, Peter and Ramasesh, Vinay and
               Webson, Albert and Pope, Reiner},
  publisher  = {Google DeepMind},
  year       = {2025},
  note       = {https://jax-ml.github.io/scaling-book/}
}
```